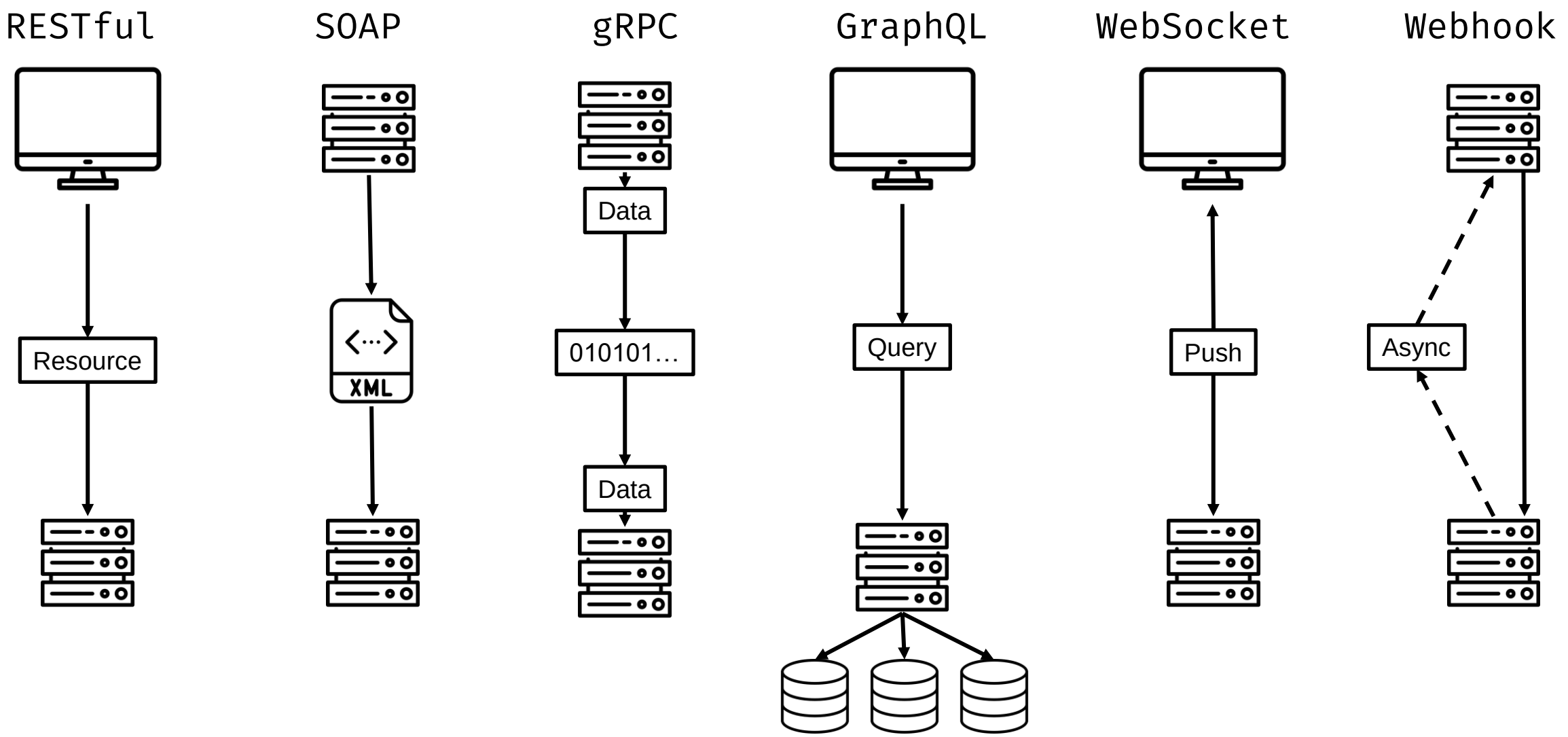
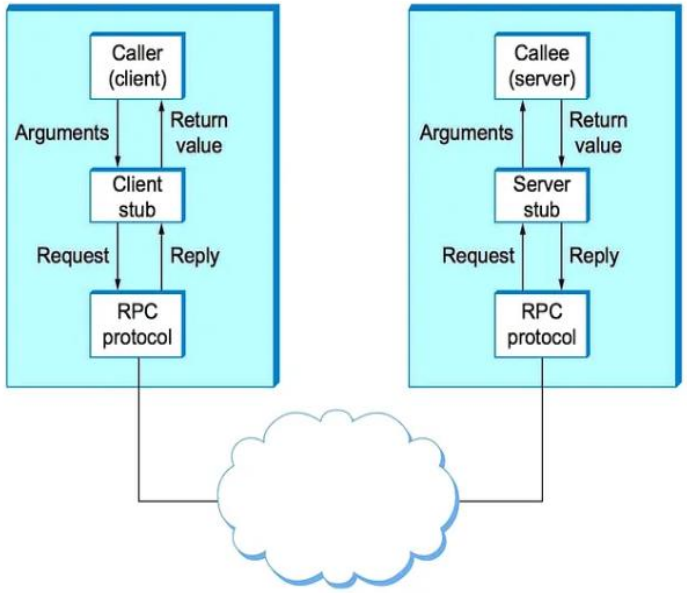
gRPC


RESTful
Resource
SOAP
XML
gRPC
Data
010101…
Data
GraphQL
Query
WebSocket
Push
Webhook
Async

gRPC = PRC + protobuf + http/2

RPC – класс технологий, позволяющих программам удаленно вызывать функции или процедуры в другом адресном пространстве



NFS Network File System — 1981, Sun

XML rpc() — 1998, Microsoft → SOAP — 2003-2007

JSON-RPC — 2005

CORBA — 1991-2021, Object Management Group

Microsoft .NET WCF — 2006-2022, Microsoft

Protocol Buffers (protobuf) – протокол сериализации (передачи) структурированных данных, разработанный Google. Protobuf предполагает описание структуры данных, которая затем компилируется в классы. Вместе с классами идет код их сериализации.

- Бинарный
- Платформо-независимый
- Быстрый
- Компактный

Компиляторы от Google:

Опенсорсные компиляторы:

```
syntax = "proto3";
option csharp_namespace = "Ozon.Route256.Seven.Protos.OrderService";

import "google/protobuf/wrappers.proto";
import "google/protobuf/timestamp.proto";
import "google/protobuf/empty.proto";
import "Protos/money.proto";


service OrdersService {
  rpc CancelOrder(google.protobuf.Int64Value) returns (CancelOrderResponse);
  rpc OrderState(google.protobuf.Int64Value) returns (OrderStateResponse);
  rpc RegionList(google.protobuf.Empty) returns (RegionListResponse);
  rpc OrdersList(OrderListRequest) returns (OrdersListResponse);
  rpc RegionAggregation(RegionAggregationRequest) returns (RegionAggregationResponse);
  rpc CustomerOrders(CustomerOrdersRequest) returns (OrdersListResponse);
  rpc OrderFullInfo(google.protobuf.Int64Value) returns (Order);
}
```

```
message RegionAggregate {
  string region = 1;
  int64 orders_count = 2;
  google.type.Money orders_cost = 3;
  double orders_weight = 4;
  int64 customers_count = 5;
}

message Order {
  uint64 id = 1;
  int64 goods_count = 2;
  google.type.Money cost = 3;
  double weight = 4;
  OrderType type = 5;
  google.protobuf.Timestamp date = 6;
  int32 region = 7;
  OrderState state = 8;
  Customer customer = 9;
  Address address = 10;
  string phone_number = 11;
}

message Customer {
  string first_name = 1;
  string last_name = 2;
}
```

```
message Address {
  string region = 1;
  string city = 2;
  string street = 3;
  string building = 4;
  string apartment = 5;
  double latitude = 6;
  double longitude = 7;
}

enum OrderState{
  CREATED = 0;
  SENT_TO_CUSTOMER = 1;
  DELIVERED = 2;
  LOST = 3;
  CANCELLED = 4;
}

enum OrderType{
  WEB = 0;
  API = 1;
  MOBILE = 2;
}
```

HTTP

HTTP/0.9

1991

- Только GET
- Заголовков нет
- Только html ответ
- Новое соединение для загрузки нового ресурса

HTTP/1.0

1996

- Добавлены POST, HEAD
- Добавлены заголовки
- Добавлены респонс коды
- Добавлено кеширование
- Добавлена авторизация
- Добавлено кодирование контента
- Ответ любого типа

HTTP/ 1.1

1999

- Добавлены PUT, DELETE, OPTIONS, TRACE, CONNECT, PATCH
- Виртуальные хосты
- Стандартизация keep-alive
- Конвейерная обработка
- Chunked для передачи потоковых данных
- Сжатие через DEFLATE

HTTP/2

2015

- Родился из гуглового SPDY
- Передача бинарных данных
- Мультиплексирование через фреймы
- Сжатие заголовков через HPACK
- Приоритезация потоков
- Передача ресурсов до запроса через Server push

HTTP/3

2022

- QUIC
- UDP вместо TCP
- Передача ресурсов до время хендшейка через 0-RTT
- Мультиплексирование быстрее
- Потоки независимы
- Server push быстрее